

BK7258 R1 视觉辅助胸牌 OpenVela 技术报告

一、需提交材料清单

序号	材料	格式 / 说明	提交要求
1	技术报告	.pdf / .docx, 按第二节模板撰写	必交
2	演示视频	≤5 分钟, mp4/mov 等; 须含功能演示、交互操作、AI 能力展示	必交
3	作品展示照片	硬件实物多角度照片 (前视图 / 后视图 / 侧视图 / 俯视图等)	可选 (涉及硬件实物则需提交)
4	海报	.pdf / .jpg 或 .pptx, 一图讲清作品 (用于线下展示)	可选 (入围决赛 / 线下展示则需提交)
5	答辩 PPT	.pptx	可选 (入围决赛提交)

- 项目源码与 AI Coding 日志均在赛事分配的专属仓 `contest2026_<编号>_<队伍名称>` 内, 无需放入压缩包; 评审将直接 clone 该仓库编译运行验证。
- 提交方式:** 将上表材料打包为一个压缩包, 在大赛官网上传。压缩包命名格式为 `<队伍名称>-<作品名称>-<仓库名称>.zip`。
示例: `不会嵌软-VelaSense-contest2026_042_buhuiqianruan.zip`

二、技术报告模板

说明: 本模板通用于各赛道, 非硬件类作品 (如纯快应用) 可跳过硬件相关小节 (如 3.4 的硬件设计与适配、材料清单中的作品展示照片等)。

1、信息表

项目	内容
作品名称	BK7258 R1 视觉辅助胸牌 OpenVela
队伍名称	不知道叫什么名字（队伍编号 441）
团队分工	邢晓亮（GitHub: xxluestc）：BSP 移植、驱动适配、构建烧录、实机验证、应用与报告整理
选题方向	新硬件平台适配

2、摘要

本项目面向 BK7258 R1 开发板，完成物理 CPU0 与 AP 侧 NuttX/openvela 的启动、跨核 RPTUN/RPMmsg、AP 双核 SMP、双屏 LCD、GC2145 摄像头和 SD 数据链路，并将拍照识别应用接入 MiMo。r14 镜像已在真实开发板烧录；实机串口记录了 KEY1 短接触发、21,099 字节 JPEG、中文识别正文及 stage=done status=0。双屏运行移植到全 OpenVela BSP 的原厂样式眼睛动画，照片展示了点亮效果，操作者已观察到眨眼。语音录音、ASR、TTS 未纳入本次提交成果；100 帧连续采集、20 次联网请求和长期稳定性未完成统计。

关键词：openvela；NuttX；BK7258；RPTUN；SMP；GC2145；MiMo；视觉辅助

3、正文

3.1 绪论

- 3.1.1 项目背景与问题定义

R1 是一块集成双屏、摄像头、麦克风、扬声器、Wi-Fi、按键、振动器、PSRAM 和 SD-NAND 的 BK7258 多核开发板。原厂方案能够运行成熟的 Armino/FreeRTOS 应用，但如果只在原厂应用上叠加识别、问答或提醒，就无法回答本项目最关心的问题：BK7258 怎样从复位入口启动 openvela，CPU0 怎样拉起 AP，多核之间如何通过 Mailbox、共享内存和 RPTUN 通信，板级初始化怎样把硬件变成 /dev 设备，音视频和网络数据又怎样在有限内存中流动。

因此，本项目把“完成 R1 的可复现全 OpenVela BSP 适配”作为主线，把图形、摄像头、音频和 Wi-Fi 当作检验 BSP 是否真实可用的载荷，而不是只追求上层功能数量。最终成果应从源码、配置、构建、镜像、Flash、CPU 启动、bring-up 到实机证据形成闭环，并能在失败时指出问题所在层级。

- 3.1.2 技术难点

****三颗物理 CPU 与两个 NuttX 层级容易混淆。**** 物理 CPU0 负责系统首启和 AP 管理；物理 CPU1 是 AP NuttX 的逻辑 CPU0；物理 CPU2 是 AP NuttX SMP 的逻辑 CPU1。只看到 AP NSH 不能证明 CPU0 也是 openvela，只看到 /proc/cpuinfo 的 processor: 0 也

不能证明 CPU2 已上线。

****启动和内存边界必须同时正确。**** CPU0、AP、bootloader、共享区、PSRAM 和用户数据有固定 Flash/RAM 契约。入口地址、VTOR、MSP、链接脚本或镜像偏移任何一处不一致，都可能表现为无日志、AP 超时或 RPTUN 长期停在 CONNECTING。

****芯片层与板级层必须分开。**** GPIO 寄存器、IRQ、SysTick 属于 BK7258 芯片层；按键、LED、屏幕复位、电源脚和 Camera 引脚属于 R1 板级映射。把板级常量写进通用 SoC 代码会破坏复用，也容易把相近开发板误当成同一硬件。

****跨核外设服务不是普通函数调用。**** Wi-Fi、Flash 等能力需要明确控制器所在 CPU、RPMsg 服务、共享缓冲所有权和错误恢复；-107、-16 等应用错误往往只是前面 AP 或服务未就绪的连锁结果。

****多媒体容易触发资源冲突。**** 两块 160×160 RGB565 帧缓冲共需 102400 字节，一帧 640×480 VYUY 图像需 614400 字节，16 kHz/16 bit/单声道音频每秒需 32000 字节，还要与网络、TLS 和 JPEG 缓冲共享 SRAM/PSRAM。

****公开参考代码不能直接等同于本队成果。**** 相同 SoC、相近板卡或他队实测只能证明候选方案，仍需固定来源提交、核对许可证、剔除产品应用，再在 R1 上完成构建和实机验证。

• 3.1.3 创新点与差异化

把 R1 从“原厂 CP + openvela AP”的混合验证推进为 CPU0 与 AP 均运行

NuttX/openvela、CPU2 进入 AP SMP 的全 OpenVela 路线，并以状态机和实机日志明确证明每一层。

建立三条互不混淆的证据基线：CPU0-only 最小 OpenVela 用于研究启动和芯片层；旧混合固件用于确认 R1 硬件；全 OpenVela 镜像用于最终架构和跨核验证。

将“源码 → 配置 → 构建 → ELF/bin → 合包 → Flash → CPU 启动 → bring-up → 设备节点/服务”做成可追溯流程，所有结论绑定提交、镜像哈希、串口日志和通过标准。

自建 bk7258-openvela-porting Skill，把来源审查、R1 差异、构建同步、失败处理和证据门禁固化为可重复流程，减少反复猜接口和无证据修改。

3.2 系统方案设计

• 3.2.1 总体架构与数据流

本次实机系统采用以下拓扑：

BootROM / Beken bootloader

|



物理 CPU0: NuttX/openvela

├─ 复位、时钟、UART、IRQ、SysTick、堆、NSH

├─ AP 生命周期管理与故障状态

└─ AP 生命周期监督、RPTUN 主端与 Wi-Fi VNET 诊断

 | Mailbox / doorbell / 共享内存 / RPTUN-RPMsg



AP NuttX/openvela

- └─ 物理 CPU1 = AP 逻辑 CPU0
- └─ 物理 CPU2 = AP 逻辑 CPU1 (SMP)
- └─ Board bring-up
- └─ framebuffer / Camera / Audio / netdev / storage
- └─ vision_badge 服务与 NSH 验证工具

主要数据流分为四条：

- 显示：应用像素或状态 → framebuffer → GC9D01 lower-half → QSPI/SPI → 两块 160×160 屏幕。
- 视觉：GC2145 寄存器配置 → I2C 与 DVP → YUV buffer/DMA → PSRAM 帧缓冲 → JPEG → V4L2 /dev/video0 → 应用服务。
- 音频：麦克风 ADC/DMA → PCM capture → 音频服务；返回音频 → PCM playback/DMA → 功放和扬声器。
- 网络：应用请求 → NuttX 网络栈 → 跨核 Wi-Fi 服务 → Beken PHY/RF；DHCP、DNS、TCP/TLS 按顺序建立。

• 3.2.2 方案论证与选型

方案	优点	局限	本项目用途
---	---	---	---
CPU0-only OpenVela	启动链最短，便于排查 UART、IRQ、SysTick、堆和 NSH	无法直接覆盖原厂多核无线与完整外设链	最小救援和芯片层学习基线，已实机通过
CPU0 Armino + AP OpenVela	原厂 Wi-Fi、多媒体和电源路径成熟，R1 硬件验证快	CPU0 实际仍是 FreeRTOS，不满足全 OpenVela 主线	只作为硬件对照和回退镜像
CPU0 OpenVela + AP OpenVela + AP SMP	系统边界统一，可完整学习启动、跨核、驱动和资源管理	移植量大，必须重做服务和外设回归	本次提交主线；实机已验证 CPU0/AP/CPU2 启动及视觉应用

R1 集成的屏幕、摄像头、音频和无线资源能够在一块板上覆盖图形、多媒体和网络三类 openvela 能力，因此选作验证平台。端侧负责启动、驱动、采集、缓存、设备控制和离线诊断；云端模型只作为可选上层能力，不承担 BSP 正确性的证明。断网或弱网时，设备仍应保留 NSH、按键、电源、显示状态和本地采集自检，不把网络失败伪装成识别结果。

• 3.2.3 关键模块设计

- ****CPU0 启动模块****：初始化 MSP/VTOR、FPU、时钟、低级串口、IRQ、SysTick 和堆，进入 nx_start()，再由 NSH 和 AP 管理应用提供可观测入口。
- ****AP 管理模块****：用明确状态记录 AP 启动、心跳、错误和 generation；失败时保留第一处错误，不让后续网络错误覆盖根因。
- ****跨核通信模块****：通过 Mailbox/doorbell 触发事件，共享内存承载数据，RPTUN/RPMmsg 管理通道生命周期；状态至少区分 CONNECTING 和 CONNECTED。
- ****SMP 模块****：AP 的逻辑 CPU0 拉起物理 CPU2，验证第二调度 CPU、IPI 和双向回调；用 online 位图和 PASSED 状态，而不是只看启动字符串。
- ****板级 bring-up****：将通用 BK7258 驱动与 R1 的引脚、电源、总线和设备映射组合，按依赖顺序注册字符设备、块设备、网络和多媒体设备。
- ****应用服务层****：vision_badge 用 camera/audio/vision/feedback 等窄接口隔离底层，

未接通的后端明确返回 `-ENOSYS`，避免把接口占位写成已完成功能。

3.3 核心算法与技术原理

• 3.3.1 启动、调度与系统节拍

复位后，链接脚本给出的向量表、初始 MSP 和程序入口必须与 Flash 中的实际镜像地址一致。bk7258_start.c 完成早期架构初始化，bk7258_lowputc.c 提供调度器启动前的串口输出，bk7258_irq.c 建立中断，bk7258_timerisr.c 配置系统节拍，最后进入 NuttX 调度。

SysTick 不只是“一个定时器”。NuttX 的延时、超时和时间片都依赖持续递增的系统 tick。项目早期出现 NSH 已能输入、但 sleep 不返回的现象，说明 UART 和命令解释器已运行，而系统节拍链未闭合。修正 SysTick 后，sleep 和定时任务恢复，这一案例用于验证“先找第一处失效的公共依赖”。

• 3.3.2 多核启动、SMP 与跨核通信

CPU0 先启动并建立共享区，再释放 AP。AP 的首核完成运行时重定位、堆和调度初始化后，与 CPU0 建立 RPTUN；随后 AP 首核通过启动入口、栈和 IPI 机制拉起物理 CPU2。三类状态含义不同：

- AP READY：AP 核心已经运行并能响应管理侧；
- RPTUN CONNECTED：两端共享内存和通知通道已经完成握手；
- CPU2 SCHEDULER_ONLINE 与 AP SMP PASSED：AP 第二调度 CPU 已上线，并完成双向调度/回调查证。

本队早期实机日志中三者同时出现且 heartbeat 递增，证明启动和跨核基本链可工作；r5 自检再次记录 AP READY、RPTUN CONNECTED、CPU2 SCHEDULER_ONLINE 与 AP SMP PASSED。

• 3.3.3 内存与资源约束

项目使用链接脚本和构建后检查共同约束 Flash 与 RAM。r14 分区中，bootloader 位于 0x00000000，CPU0 应用位于 0x00011000，AP 位于 0x00143000，pair 位于 0x002ca000；EasyFlash、RF 校准和用户数据分区不被本次局部更新覆盖。AP 运行区与 CPU0/SWAP 边界同样需要检查，不能只看 bin 文件大小。

大缓冲优先放入 PSRAM，DMA 缓冲还要满足硬件可访问范围、对齐和缓存一致性。两块显示帧缓冲、VGA 图像、JPEG 工作区、音频环形缓冲和 TLS 内存不能各自按峰值无限申请，终版应以实际 map、free 和重复开关测试决定预算。

3.3.4 openvela 设备模型与多媒体机制

NuttX 通过 upper-half/lower-half 把应用接口与硬件实现分离。板级 bringup 调用注册函数后，应用经文件描述符和 ioctl 使用 /dev/fb*、/dev/video*、/dev/audio/*、/dev/mmc* 等设备，而不是直接调用板级寄存器函数。这样可把 GC2145、GC9D01 或音频控制器的 R1 差异留在 lower-half 和 board 层，并让应用服务保持稳定。

旧混合固件已用这套路径证明 R1 硬件链：GC2145 经 DVP/DMA 采集到 PSRAM，再由 JPEG

编码；PCM capture/playback 已完成 3 秒回环；双 framebuffer 可驱动两块圆屏。全 OpenVela BSP 的 r13/r14 又记录了 Camera JPEG、SD 数据面和原厂样式眼睛线程；本次语音应用闭环未完成。

3.3.5 AI 能力边界

本项目没有在 BK7258 上部署端侧大模型，AI 能力采用云端 MiMo 图片理解。r14 实机已完成 Camera JPEG、Wi-Fi/DHCP、HTTPS、图片上传、JSON 解析、跨核返回和中文结果打印；KEY1 可触发这一链路。API Key 从设备侧配置加载，不写入镜像和普通日志。语音 ASR、TTS 和语音播放未纳入本次成果。

3.4 系统实现

- 3.4.1 软件与固件结构

当前队伍仓按职责分为:

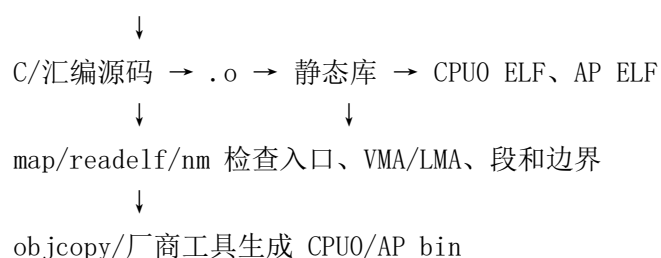
chips/bk7258/ 器驱动	CPU0/AP/CPU2、IRQ、SysTick、RPTUN 与控制
boards/bk7258/common/ boards/bk7258/aidk_ai_toy/ nuttx/	通用启动、bring-up、链接脚本和分区规则 R1 引脚、外设、电源与本队 CP/AP 配置 本 BSP 依赖的 NuttX 覆盖文件
tools/bk7258/ app/bsp_diag/ app/vision_badge/	双镜像构建、地址检查、镜像编码和打包工具 AP/RPTUN/SMP 与 Wi-Fi VNET 诊断命令 Camera、MiMo、RPC 与反馈应用
.agents/skills/bk7258-openvela-porting/ evidence/ logs/	移植审查 Skill、审计脚本和规则 脱敏后的本队实机证据 大赛 AI Coding 原始日志

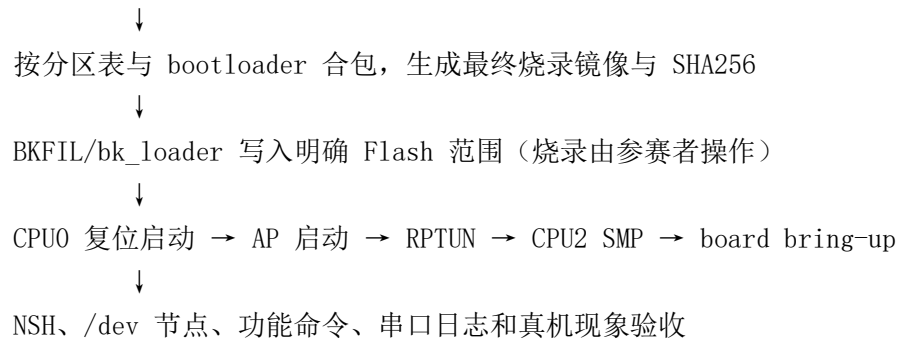
全 OpenVela BSP 主参考固定为 <https://github.com/openvela/contest2026> 135 yongwangzhiqian 的

7079493e73159e00c1b03e60bee6ee69845751cd, 根许可证为 Apache-2.0; Beken SDK 固定为 Embracecactus/bk_avdk_smp@cb080de1655d579c7593ecf504c440997c4c137b。本队只整合 CPU0/AP 启动、SMP、RPTUN、R1 board、Wi-Fi/Flash 服务和构建打包文件, 不复制对方产品应用、UI、云端业务、模型和凭据。文件级边界记录在 SOURCE PROVENANCE.md。

- 3.4.2 从源码到实机的完整流程

Kconfig/defconfig 选择功能





烧录地址不是从文件名推测，而是由分区表、链接地址和打包清单共同确定。项目曾用局部写入只更新 CPU0/AP，避开 boot、配对、EasyFlash、RF 校准和用户数据区域；每个候选镜像都应记录输入文件、地址、长度、工具版本、日志和 SHA256，并保留已验证的回退镜像。

- 3.4.3 芯片层与板级适配
- bk7258_start.c: 复位入口与进入 NuttX 前的架构初始化；
- bk7258_lowputc.c、bk7258_serial.c: 早期 UART 和 NuttX 串口设备；
- bk7258_irq.c: 中断控制器初始化与 IRQ 分发；
- bk7258_timerisr.c: 系统节拍；
- bk7258_allocateheap.c: 堆边界；
- bk7258_gpio.c: 与 SoC 相关的 GPIO 操作；
- boards/bk7258/aidk_ai_toy/src/: R1 早期板级初始化、驱动注册与设备 bring-up；
- bk7258_buttons.c、bk7258_userleds.c: R1 引脚映射与 NuttX board API。

GPIO 实测曾发现初始 LED 引脚映射错误：KEY1 和 KEY3 只能点亮同一颗绿灯。回到 R1 原理图和板级映射修正后，KEY1/KEY2/KEY3 分别产生掩码 0x01/0x02/0x04，并对应红灯、绿灯和双灯。这说明不同 board 的引脚必须留在板级，不能通过猜测修改通用 GPIO 驱动。

• 3.4.4 应用和交互实现

vision_badge 把应用拆为 camera、vision、feedback、workflow 和 RPC 服务。CP 的 NSH 命令或 KEY1 短按通过 RPMsg 触发 AP 采图，AP 使用 V4L2 获取 JPEG、通过 HTTPS 请求 MiMo 并把有界中文结果返回 CP。r14 的 KEY1 路径在 CP 串口打印中文正文。语音录音、ASR、TTS 仍保持为后续项。

本次人机交互以 KEY1 短按拍照识别和双屏眼睛动画为主。r13 启动日志记录 AIDK EYE START vendor dual-eye animation 和眼睛任务调度；r14 沿用同一 AP 镜像，硬件照片记录双屏点亮，操作者观察到眨眼。KEY1 长按关机/唤醒属于旧混合固件验证结果，本次不列为 r14 功能。

• 3.4.5 自建 Skill

自建 Skill 位于 .agents/skills/bk7258-openvela-porting/，不是一段泛化提示词，而是面向本项目的移植流程：

读取来源图，区分上游、厂商、公开候选和本队实测；

检查 R1 原理图/BOM、许可证、固定 commit 和板级差异；

用同步脚本保证队伍仓库板级代码与完整 openvela 工作区一致；
用审计脚本检查项目结构、配置、证据和占位后端；
以“源码存在 → 配置生效 → 构建通过 → 镜像正确 → R1 实机通过”为证据门禁；
对失败记录第一处异常、层级、回退点和下一实验。

仓库现有 Python 回归测试用于检查 Skill 审计规则。自建 Skill 是开发流程工具，不属于设备运行时 Agent Skill；本次参赛方向为新硬件平台适配。

3.5 系统测试与结果分析（务必提供量化数据）

• 3.5.1 测试环境

硬件为 BK7258 R1/AIDK AI Toy，串口 COM12，115200 8N1。固件为 r14，CP/AP 全 OpenVela，AP SMP=2。构建在 Ubuntu VM 通过 BK7258 direct build 完成，镜像哈希见 evidence/final/vision-badge-r14-SHA256SUMS.txt。MiMo 模型为 mimo-v2.5。实机证据集中在 evidence/final/，包括构建清单、烧录日志、KEY1 和命令触发的查询日志。

• 3.5.2 功能测试结果

CP 到 AP RPC 返回 AP endpoint ready。r13 同 AP 镜像启动日志记录 Wi-Fi DHCP ACK、Camera 数据面通过、SD 数据面通过和原厂样式眼睛任务启动。r14 的 KEY1 实测输出 VISION BADGE KEY1 short press; starting capture，采集 21,099 字节 JPEG，返回 81 字节中文结果，最终为 stage=done status=0。识别正文为“一位戴眼镜的年轻男子，耳朵里塞着白色耳机，正对着镜头。”

r14 另一条命令触发的实测查询采集 17,527 字节 JPEG，返回 78 字节中文结果，stage=done status=0。因此现有保存证据包含两次成功查询，其中一次由 KEY1 触发。语音录音、ASR、TTS 未接入本次应用闭环。

• 3.5.3 性能和资源数据

本次 r14 保存日志中的实机视觉请求次数为 2；JPEG 样本大小分别为 17,527 和 21,099 字节；MiMo 结果长度分别为 78 和 81 字节；两次均为 stage=done status=0。AP 镜像为 813,858 字节，CP 镜像为 920,754 字节，pair 镜像为 2,854,912 字节。Camera 配置为 640×480 JPEG；RPMsg 文本槽为 256 字节。

100 帧连续采集、20 次联网请求统计、功耗、长期运行和峰值内存未测量。这两次成功查询不外推为完整稳定性百分比。

• 3.5.4 可靠性与异常处理

源码含 Camera 参数和 JPEG 标记校验、MiMo network/clock/credentials/http/response 阶段日志、HTTP 非 2xx、JSON 缺字段、超大结果和 RPC endpoint 未就绪处理。r7 的 DNS 失败被定位为 AP DNS 配置缺失，r8 修复后取得 HTTPS 链路；r14 补齐 KEY1 成功路径的中文正文打印。

10 次冷启动统计、20 次请求成功率、100 帧 Camera 压力、60 秒音频、长期运行、功耗和断电恢复尚未完成，因此本报告只宣称当前保存的功能结果。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	未建立逐行归因口径，不填虚构百分比
使用的 AI 工具	Codex、MiMo Code、本地终端、源码检索与文档处理
MCP 工具使用情况	未使用 VelaJS MCP 或 Figma MCP
Skills 使用与新增情况	bk7258-openvela-porting；用于来源审查、分层、构建与证据门禁，12 项测试通过
Token 使用总量	日志 3 个会话、76 条事件；无 Token 字段，未虚构总量

3.6.1 AI 工具使用

AI Coding 用于检索 NuttX、Beken 和 OpenVela 源码，起草板级 glue 与应用接口，分析串口错误码，整理镜像清单，生成回归检查并编写技术报告。GPIO、Flash 地址、设备节点、构建结果和识别结果均由源码、VM 构建日志或 R1 串口日志复核。仓库内 Mimocode 日志包含 3 个会话、76 条事件，manifest 校验为 ok；日志没有可追溯 Token 字段，因此不虚构总量。

3.6.2 AI 工具指标

AI Coding 代码占比未建立逐行归因口径，不填虚构百分比。使用工具包括 Codex、MiMo Code、本地终端、源码检索、文档处理和网页资料核对。未使用 VelaJS MCP 或 Figma MCP。自建 Skill 为 bk7258-openvela-porting，项目审计 12 项全部通过。硬件实测和最终提交字段由人工确认，AI 输出不替代 R1 证据。

3.6.3 典型问题与解决方式

Wi-Fi 初次查询在 TLS 阶段返回 -82，通过阶段日志定位为 DNS client 缺失；加入 DNS client 后重新构建并在 R1 获得 MiMo HTTP 成功。MiMo 长回答曾被固定 RPC 槽截断，通过限制提示词长度和 UTF-8 安全拷贝修正。参考仓只用于 BSP 对照，产品应用、云端业务和凭据没有直接复制。

3.7 总结与展望

3.7.1 成果总结

本项目完成 BK7258 R1 的全 OpenVela CP/AP 架构、AP SMP、RPTUN/RPMsg、双 LCD、GC2145 V4L2 JPEG、SD 数据面和 MiMo 视觉应用闭环。r14 在真实开发板上完成 KEY1 拍照识别并打印中文结果，另有一次 NSH 命令触发的成功查询。构建清单、镜像 SHA256、烧录日志和串口证据收录在仓库 evidence/final/。

3.7.2 应用前景与商业价值

视觉辅助胸牌适合视障用户在室内、校园和日常通行场景中获得简短环境提示。当前成果

更适合作为 BK7258 R1 的 OpenVela 参考工程和视觉辅助产品原型：端侧提供可复用的 Camera、LCD、网络 and 跨核基础，云端模型按需提供图片理解。后续商业化仍需用户调研、功耗评估、隐私策略、离线能力和语音交互验证，本文不虚构市场规模和商业收入。

• 3.7.3 不足与后续工作

录制不超过 5 分钟的功能演示视频，并补充硬件背面与侧面照片。

完成 10 次冷启动、20 次视觉请求、100 帧连续 Camera 和长期运行统计。

增加 JPEG 主机端独立解码、内存高水位、功耗和断电恢复证据。

在不影响当前视觉闭环的前提下恢复麦克风、ASR、TTS 和语音播报。

按官方要求打包技术报告、视频和照片，保留 API Key、Wi-Fi 密码和设备私有数据在提交包之外。

评审维度（分值）	报告对应章节
技术难度（30）	3.2 系统方案设计；3.3 核心算法与技术原理；3.4 系统实现；3.5 系统测试
产品创新性（20）	2、摘要；3.1 绪论（创新点）
项目完整度（20）	3.5 系统测试与结果分析；项目源码；作品展示照片
AI 开发（10）	3.3 核心算法（AI 算法实现）；3.6 AI-Native 开发说明
商业潜力（10）	3.7 总结与展望（应用前景与商业价值）
展示效果（10）	演示视频；海报；答辩 PPT

四、注意事项

- 作品提交截止：**2026 年 9 月 20 日**，请合理安排开发与提交节奏。
- AI Coding 日志由插件自动写入仓内 logs/ 目录，请勿手动删改。
- 作品须为原创，遵循 Apache 2.0 开源协议，不得存在版权、专利及法律纠纷。
- 项目须使用 openvela 系统能力（Nuttx 内核仓库除外），且至少落地图形、AI、多媒体三项核心能力之一。
- 若含 AI 语音唤醒功能，统一使用唤醒词：你好，openvela / Hello, openvela。